

FAQ Chatbot - Boss Presentation Prep Guide

Complete guide: explain from start to end + 18 Q&A; | Pavan Adithya

IMPORTANT: Read tonight. Practice each answer OUT LOUD at least once.

SECTION 1 - How to Open and Start (Do this FIRST)

Step 1: Open VS Code > File > Open Folder > select FAQCHATBOT

Step 2: Open Terminal in VS Code (Ctrl + tilde key)

Step 3: Type: start_all.bat and press Enter

Step 4: Wait 10 seconds - two windows open automatically

Step 5: Open Chrome > go to <http://localhost:3000>

Step 6: Type any question in the chat to show it works live

Say to your boss:

"This is our FAQ Chatbot running locally. Backend is on port 8000 and frontend on port 3000."

SECTION 2 - 60-Second Project Introduction

Say exactly this:

"I built a FAQ Chatbot using a 9-step AI pipeline. When a user asks a question, the system does NOT do a simple keyword match. Instead, it converts the question into a 384-dimensional vector using an AI embedding model called all-MiniLM-L6-v2, searches a vector database called Pinecone for the most semantically similar FAQ, and generates a natural language answer. If nothing matches above the 0.50 confidence threshold, it gives a polite fallback with 3 suggested questions."

SECTION 3 - Architecture

Open CODE_STRUCTURE.md in VS Code. Say: "The project has two main parts:"

Part	Technology	Port
Backend	Python + FastAPI + Uvicorn + Pinecone + SQLite	8000
Frontend	Next.js 14 + React 18 + TypeScript + Tailwind CSS	3000

"The backend has 9 service files - each handles one step of the pipeline."

SECTION 4 - The 9-Step Pipeline (MOST IMPORTANT)

Point to each file when explaining:

Step	What Happens	File
Step 1	All 50 FAQs loaded into Pinecone at startup	ingest.py
Step 2	User question converted to 384-number vector (embedding)	embeddings.py
Step 3	Validate - is the question empty or too short?	main.py
Step 4	Route - FAQ question / Greeting / Out of scope?	query_router.py
Step 5	Search Pinecone - top 3 most similar FAQs	semantic_search.py
Step 6	Best match score >= 0.50? Pick best answer	answer_selector.py

Step	What Happens	File
Step 7	Rewrite vague words like 'it', 'my order' to actual topic	query_rewriter.py
Step 8	Write clean natural language answer	response_writer.py
Step 9	If no match - polite fallback + 3 suggested questions	fallback.py

Tip: Open backend/main.py lines 96-106 - the 9 steps are listed as comments. Show them!

SECTION 5 - Data, ENV and API Keys

"We have 50 real-world FAQs in backend/data/faq_data.json across 9 business categories. Every chat is saved to SQLite by db_service.py. All API keys are in a .env file - never pushed to GitHub. Only .env.example is visible on GitHub as a template."

ENV Variable	Purpose	Required?
PINECONE_API_KEY	Connects to Pinecone vector database	YES (free tier)
PINECONE_ENVIRONMENT	Pinecone region (e.g. us-east-1)	YES
OPENAI_API_KEY	Optional LLM for richer answers	NO - Optional
GROQ_API_KEY	Optional fast free LLM	NO - Optional
GEMINI_API_KEY	Optional Google AI model	NO - Optional
SIMILARITY_THRESHOLD	Confidence cutoff (default 0.50)	Has default
EMBEDDING_MODEL	AI model name (all-MiniLM-L6-v2)	Has default
PORT	Backend server port (default 8000)	Has default

SECTION 6 - API Endpoints

Method	URL	What it does
POST	/api/chat	Send question, get answer - main.py line 94
GET	/api/faqs	Get all 50 FAQs - main.py line 75
GET	/api/health	Check if backend is running - main.py line 65
POST	/api/ingest	Re-load FAQs into Pinecone - main.py line 83
GET	/api/history	Get recent chat logs (SQLite) - main.py line 90

SECTION 7 - Live Demo Questions

Type these questions in the chatbot during the demo:

Type this	What it shows
"How do I reset my password?"	Normal FAQ answer with high confidence
"Where is my order?"	Order/shipping category answer
"What is the weather today?"	Out-of-scope - fallback with 3 suggestions
"Hi"	Greeting route - friendly welcome message
"asdfjkl"	Random text - low confidence fallback

SECTION 8 - Technologies and Versions

Technology	Version	Purpose
Python	3.10+	Backend language
FastAPI	>= 0.110.0	REST API framework
Uvicorn	>= 0.28.0	Python web server
Pydantic	>= 2.6.0	Data validation
python-dotenv	>= 1.0.1	Reads .env file
Pinecone	>= 3.1.0	Vector database client
Sentence-Transformers	>= 2.5.1	Local AI embedding model (all-MiniLM-L6-v2)
PyTorch	>= 2.0.0	AI framework (powers embeddings)
NumPy	>= 1.24.0	Math and vector operations
SQLite	Built-in	Chat history database - no setup needed
Node.js	18+	Frontend runtime
Next.js	14	React full-stack framework
React	18	UI component library
TypeScript	5	Type-safe JavaScript
Tailwind CSS	3	Utility-first CSS styling

SECTION 9 - 18 Questions Your Boss Will Ask

Read ALL answers tonight. Say each green answer OUT LOUD once.

Q1: What is this project about?

"It is an AI-powered FAQ Chatbot. Instead of keyword matching, it uses semantic search - it understands the MEANING of the question and finds the closest answer from our FAQ database using vector similarity."

Q2: What technologies did you use?

"Backend: Python, FastAPI, Uvicorn, Pinecone, Sentence Transformers, SQLite. Frontend: Next.js 14, React 18, TypeScript, Tailwind CSS. The AI embedding model is all-MiniLM-L6-v2 - it runs locally with no API key needed."

Q3: What is Pinecone?

"Pinecone is a vector database. Normal databases store text - Pinecone stores numbers that represent the meaning of text. When I search, it finds the most semantically similar FAQ, not just one with matching words."

Q4: What is an embedding?

"An embedding converts a sentence into a list of numbers - in our case, 384 numbers. These numbers capture the meaning of the text. Similar meanings = similar numbers. That is how semantic search works."

Q5: What is the similarity threshold?

"It is set to 0.50. If the best Pinecone match scores above 0.50 out of 1.0, we are confident and give the answer. Below 0.50, we fall back to a polite response with 3 suggested questions."

Q6: Why FastAPI instead of Flask?

"FastAPI is faster, has automatic API docs at /docs, supports async, and validates data with Pydantic automatically. It is the modern standard for Python REST APIs."

Q7: Why Next.js for frontend?

"Next.js 14 gives both a UI and serverless API routes in one framework. For cloud deployment, the frontend runs the full 9-step pipeline without needing a separate Python server."

Q8: What if the user asks something not in the FAQs?

"Step 4 (Query Router) first checks if it is out of scope. If it passes but scores below the threshold in Step 6, Step 9 (Fallback Handler) gives a polite response and suggests 3 relevant questions."

Q9: What is the query rewriter?

"Users say vague things like 'Where is it?' The query rewriter maps common vague phrases to specific topics. For example, 'my order' becomes 'order status and delivery'. This improves Pinecone search accuracy."

Q10: How is data stored?

"Two ways. The 50 FAQs are in `faq_data.json` and indexed into Pinecone as vectors at startup. Every chat - the question and answer - is saved to a local SQLite database by `db_service.py`."

Q11: Is the .env file on GitHub?

"No. It is in `.gitignore`. Only `.env.example` is on GitHub as a template showing what keys are needed. The actual key values stay only on my local machine. This is standard security practice."

Q12: How do you run the project?

"Just double-click `start_all.bat` - it starts both the Python backend and Next.js frontend in one click. No need to open two separate terminals."

Q13: What are the API endpoints?

"Five endpoints: `POST /api/chat` (question to answer), `GET /api/faqs` (list all FAQs), `GET /api/health` (server check), `POST /api/ingest` (re-index FAQs), `GET /api/history` (chat logs)."

Q14: What is CORS and why did you enable it?

"CORS means Cross-Origin Resource Sharing. Frontend runs on port 3000 and backend on port 8000. The browser blocks cross-port requests by default. I enabled CORS in FastAPI to allow the frontend to communicate with the backend."

Q15: Did you test the project?

"Yes. I wrote 9 automated tests in `test_pipeline.py`. They test embedding generation, Pinecone ingestion, similarity search, fallback logic, and the full end-to-end pipeline. All 9 tests pass."

Q16: Can this be deployed to the cloud?

"Yes. I created `render.yaml` for the Python backend on Render.com and `vercel.json` for the Next.js frontend on Vercel. The frontend also has its own serverless API routes for fully cloud-native deployment."

Q17: Why SQLite and not PostgreSQL?

"SQLite is built into Python with zero setup needed. For demo and development it is perfect. For production, we can swap to PostgreSQL easily - only `db_service.py` needs to change, the rest of the code stays the same."

Q18: Difference between semantic search and keyword search?

"Keyword search matches exact words. Semantic search matches MEANING. Example: 'How do I get into my account?' and 'I forgot my login' have zero matching keywords, but semantic search knows they mean the same thing and returns the same FAQ."

GOLDEN RULE - If you do not know an answer

"That is a good point - I would need to check the implementation details and get back to you on that."

NEVER say: I don't know. NEVER go silent. Stay calm. Smile. You built this - you know it!

QUICK CHEAT SHEET - Numbers to Memorize

Item	Value
Number of FAQs	50
FAQ categories	9
Pipeline steps	9
Automated tests	9 (all passing)
Similarity threshold	0.50
Embedding dimensions	384
Embedding model	all-MiniLM-L6-v2
Backend port	8000
Frontend port	3000
API endpoints	5
Backend framework	FastAPI
Frontend framework	Next.js 14
Vector database	Pinecone
Chat history database	SQLite
GitHub (boss repo)	github.com/revalsysprashant/FAQ-Chatbot

TIP: Start the project BEFORE your boss arrives. Have the browser open on <http://localhost:3000>. Show the live demo first - it makes everything easier to explain.